

Computernetze und ihre Leistung Assignment 2

Einführung 1 cuil-lab – Netzwerk-Lab in Docker

Für diese Übung verwenden Sie *cuil-lab*, eine kleine Terminal-Anwendung, die Ihnen ein vollständiges Versuchsnetz aus Docker-Containern aufbaut: zwei Hosts (**n1** als Sender, **n2** als Empfänger), die über einen *Shaper*-Container verbunden sind. Auf dem Shaper wendet Linux **tc** Bandbreite, Verzögerung, Jitter und Paketverlust auf den Link an, die Sie zur Laufzeit ändern können. Die Hosts enthalten alle Werkzeuge, die Sie für die Aufgaben benötigen (*iperf3*, *tcpdump*, **ss** sowie eigene UDP- und TCP-Clients und -Server). Sie müssen also keine eigene Software implementieren, sondern die mitgelieferten Programme aufrufen.

Installation: Bitte führen Sie die folgenden Schritte vor Beginn der Aufgabenbearbeitung einmalig durch. *cuil-lab* läuft lokal auf Ihrem Rechner und benötigt *Docker* sowie *Python 3.11+*.

Windows: Nutzen Sie eine **Linux-VM**:

- Installieren Sie eine Virtualisierungslösung, z. B. *VirtualBox* (<https://www.virtualbox.org/>) oder *VMware Workstation Player*.
- Richten Sie darin eine *Ubuntu 24.04 LTS*-VM ein (offizielles Ubuntu Desktop Image, <https://ubuntu.com/download/desktop>). Reservieren Sie der VM ausreichend Ressourcen (mindestens 2 CPU-Kerne, 4 GB RAM).
- Führen Sie in der VM anschließend die Installationsschritte für *Linux (Ubuntu/Debian)* unten aus.
- Achten Sie darauf, dass die VM über NAT oder Bridged Networking Internetzugriff hat (für **apt** und den Image-Build).

macOS:

- Installieren Sie *Docker Desktop* (<https://docs.docker.com/get-docker/>) und starten Sie es.
- Python 3.11+ erhalten Sie z. B. über Homebrew: **brew install python**.

Linux (Ubuntu/Debian):

- Installieren Sie Docker und das Python-venv-Modul:
`sudo apt install docker.io python3 python3-venv`
- Fügen Sie Ihren Nutzer der Gruppe **docker** hinzu, damit Sie Docker ohne **sudo** verwenden können:
`sudo usermod -aG docker $USER`
Melden Sie sich anschließend einmal ab und wieder an.

Lab starten:

- Laden Sie die Materialien (das Übungsverzeichnis mit dem Skript **run-lab.sh** und dem Ordner **docker/**) aus dem Learnweb-Bereich der Veranstaltung herunter.
- Starten Sie das Lab im Übungsverzeichnis mit **./run-lab.sh**. Beim ersten Aufruf baut das Skript automatisch die Docker-Images und richtet eine Python-Umgebung ein; das kann einige Minuten dauern. Danach öffnet sich das *cuil-lab*-Dashboard im Terminal.
- Das Dashboard zeigt die Topologie und Live-Statistiken. Über Tastenkürzel steuern Sie das Lab, u. a.:

- **l** – Linkparameter (Bandbreite, Delay, Jitter, Loss) ändern; beim Anwenden werden immer alle Felder gesetzt, prüfen Sie daher vor dem Anwenden alle Werte im Dialog
- **c** – TCP-Staukontrolle eines Hosts setzen
- **i** – iperf3-Test starten
- **u / t** – UDP- bzw. TCP-Werkzeug starten (Logdateien landen im Ordner **docker/out/**)
- **p** – Paketaufzeichnung an-/ausschalten; es entsteht je ein Mitschnitt pro Endpunkt des Links in **docker/out/** (**cap-<Zeitstempel>-n1.pcap** und **...-n2.pcap**)
- **s** – Shell auf einem Host öffnen (geteilte Ansicht; eine Shell pro Host, mehrere gleichzeitig möglich)
- **Strg-0** – Fokus zyklisch zwischen Dashboard und offenen Shells wechseln. Hat eine Shell den Fokus, gehen alle Tasten an die Shell; mit **Strg-0** erreichen Sie wieder die Tastenkürzel des Dashboards.
- **Strg-W** – fokussierte Shell schließen (alternativ **exit** eingeben)
- **r** – zurücksetzen, **q** – beenden, **?** – Hilfe
- Jede Shell startet im Verzeichnis **/node**, dem privaten Arbeitsbereich des jeweiligen Hosts. Auf Ihrem Rechner liegt er als **docker/nodes/<host>/**; Dateien sind auf beiden Seiten sichtbar (praktisch für eigene Skripte und gesicherte Messausgaben).
- Testen Sie die Installation: Öffnen Sie mit **s** eine Shell auf **n1** und prüfen Sie mit **ping n2** die Grundkonnektivität.

Aufgabe 2 Analyse von Retransmissions und Flusskontrolle im Netzwerkverkehr

Im Rahmen dieser Aufgabe analysieren Sie das Verhalten von UDP- und TCP-Kommunikation in der **cuil-lab**-Umgebung mit Fokus auf Retransmissionen und Flusskontrolle. Sie nutzen die mitgelieferten UDP- und TCP-Werkzeuge der Hosts, beobachten detailliert die Netzwerkaktivitäten unter normalen und verlustbehafteten Bedingungen und berichten Ihre Ergebnisse.

Aufbau:

- Starten Sie das Lab mit **./run-lab.sh**. Die Topologie steht bereit: zwei Hosts **n1** (Sender) und **n2** (Empfänger), über einen Shaper verbunden.
- Der Link ist anfangs ohne Verlust oder Verzögerung konfiguriert (Standardwerte aus **lab.yaml**). Mit **l** ändern Sie die Linkparameter zur Laufzeit.
- Eine manuelle IP-Konfiguration ist nicht nötig; Adressierung und Routing über den Shaper richtet das Lab automatisch ein. Die Hosts sind über die Namen **n1** und **n2** erreichbar.
- Zeichnen Sie den Verkehr mit **p** auf. Es entstehen zwei PCAP-Dateien im Ordner **docker/out/** (**docker/out/cap-<Zeitstempel>-n1.pcap** und **...-n2.pcap**), je eine pro Endpunkt; beide lassen sich mit Wireshark öffnen. Verlorene Pakete erkennen Sie daran, dass sie im Mitschnitt des Senders (**n1**) auftauchen, im Mitschnitt des Empfängers (**n2**) aber fehlen.
- Prüfen Sie die Grundkonnektivität bei Bedarf mit **ping n2** aus einer Shell (**s**).

a) UDP-Verkehr und Verlust

- **Schnelltest (optional):**

Für einen schnellen Funktionstest können Sie im Dashboard mit **u** einen kurzen UDP-Lauf starten oder die Grundkonnektivität mit **ping n2** prüfen.

- **Werkzeuge:** Öffnen Sie mit **s** eine Shell auf **n2** und starten Sie dort den mitgelieferten UDP-Server, auf **n1** den UDP-Client. Der Client sendet in regelmäßigen Abständen Nachrichten an den Server, ohne ein eigenes Verlustrecovery zu implementieren (UDP ist verbindungslos). Alternativ startet die Taste **u** beide Programme automatisch.

auf n2:

```
udp_server.py --port 9999
```

auf n1:

```
udp_client.py --host n2 --port 9999 \  
--interval 1 --count 30
```

- **Ablauf:**

- Starten Sie *zuerst* die Paketaufzeichnung mit **p** (PCAPs in **docker/out/**), damit auch die ersten Pakete erfasst werden.
- Führen Sie einen ersten Lauf unter *normalen* Bedingungen durch (kein Verlust, Standardwerte): Server auf **n2**, Client auf **n1** starten.
- Setzen Sie anschließend auf dem Link 30 % Paketverlust: Taste **l**, Feld **loss** auf 30%. Der Verlust wirkt in beiden Richtungen unabhängig; es können also auch Antworten des Servers verloren gehen.
- Führen Sie einen zweiten Lauf unter *verlustbehafteten* Bedingungen durch.
- Stoppen Sie die Paketaufzeichnung wieder mit **p**.

- Analysieren Sie die Paketausfälle anhand der PCAP-Dateien, insbesondere:

- Welche Nachrichten gehen verloren bzw. werden nicht empfangen?
- Wie reagiert der UDP-Client auf Paketverluste?
- Diskutieren Sie die fehlende Retransmission bei UDP. Beschreiben Sie dabei konkret, welche Auswirkungen dies auf Anwendungsprotokolle und deren Zuverlässigkeit haben kann und welche Strategien auf Anwendungsebene genutzt werden könnten, um mit Paketverlusten umzugehen.

- Reichen Sie einen kurzen Bericht mit Ihren Beobachtungen und den verwendeten Aufrufparametern ein.

b) TCP-Verkehr: Retransmissionen und Flusskontrolle

- **Schnelltest (optional):**

Für einen schnellen Funktionstest können Sie im Dashboard mit **t** einen kurzen TCP-Lauf starten oder die Grundkonnektivität mit **ping n2** prüfen.

- **Werkzeuge:** Öffnen Sie mit **s** eine Shell auf **n2** und starten Sie dort den mitgelieferten TCP-Server, auf **n1** den TCP-Client. Der Client unterstützt zwei Betriebsarten:

- *persistent*: Wiederverwendung einer einzigen TCP-Verbindung für alle Nachrichten.
- *reconnect*: Aufbau einer neuen TCP-Verbindung für jede einzelne Nachricht.

auf n2:

```
tcp_server.py --port 9999
```

auf n1, Modus persistent:

```
tcp_client.py --host n2 --port 9999 \  
--interval 1 --count 30
```

```
--interval 1 --count 30 --mode persistent
```

```
# auf n1, Modus reconnect:
```

```
tcp_client.py --host n2 --port 9999 \  
--interval 1 --count 30 --mode reconnect
```

- **Ablauf:**
 - Starten Sie *zuerst* die Paketaufzeichnung mit **p** (PCAPs in **docker/out/**).
 - Führen Sie die Läufe (beide Modi) zunächst unter *normalen* Bedingungen durch.
 - Setzen Sie anschließend auf dem Link 30 % Paketverlust (Taste 1, Feld **loss** auf 30%).
 - Wiederholen Sie die Läufe (beide Modi) unter *verlustbehafteten* Bedingungen.
 - Stoppen Sie die Paketaufzeichnung wieder mit **p**.
- Beobachten Sie die Fenstergrößen im laufenden Betrieb in einer Shell (**s**) mit **ss -tin**; das Dashboard zeigt zusätzlich **Senderate**, **cwnd** und **srtt** live an. Ein Beispiel zur fortlaufenden Protokollierung finden Sie in den Hinweisen.
- Analysieren Sie mit dem Fokus auf:
 - Erkennung und Verhalten von Retransmissions unter Verlust.
 - Funktionsweise der Flusskontrolle (Empfangsfenster **rwin**).
 - Vergleich der beiden Modi (Verbindungsrecycling vs. Verbindungsaufbau für jede Nachricht) hinsichtlich Overhead, Latenz und Stabilität bei Paketverlust.
 - Verzichten Sie auf detaillierte Analyse der Staukontrolle (congestion control); Fokus liegt auf Flusskontrolle und Fehlerkorrektur.
- Reichen Sie einen ausführlichen Bericht mit PCAP-Analyse, Diagrammen der Empfangsfenster und den verwendeten Aufrufparametern ein.

c) TCP-Verhalten unter Verzögerungen (Jitter)

- Setzen Sie den Link zunächst mit `r` zurück bzw. stellen Sie sicher, dass der Paketverlust aus der vorherigen Teilaufgabe wieder auf 0% steht – andernfalls überlagern sich Verlust- und Jitter-Effekte. Beachten Sie, dass beim Anwenden über 1 stets alle Felder gesetzt werden.
- Setzen Sie auf dem Link über die Taste 1 eine Grundverzögerung mit Schwankung, z. B. `delay` auf 20ms und `jitter` auf 10ms. Der Jitter wirkt als zufällige Schwankung um die Grundverzögerung; wählen Sie die Grundverzögerung daher größer als den Jitter.
- Führen Sie das TCP-Experiment erneut mit `tcp_server.py/tcp_client.py` durch (Aufzeichnung mit `p` vor dem Start aktivieren).
- Messen Sie die Paketankunftszeiten und beobachten Sie das Verhalten der Flusskontrolle.
- Analysieren Sie insbesondere, wie der Empfangspuffer (Empfangsfenster) auf Schwankungen der Paketankunftszeiten reagiert.
- Stellen Sie relevante Diagramme dar und diskutieren Sie die Auswirkungen von Jitter auf die Flusskontrolle und Retransmissionsrate von TCP.
- Reichen Sie einen kurzen Analysebericht mit Grafiken und Erläuterungen ein.

Hinweise:

- Die Experimente nutzen die mitgelieferten Python-Werkzeuge der Hosts; im Dashboard starten die Tasten `u/t/i` sie auch direkt.
- Die Werkzeuge schreiben auf die Standardausgabe. In einer Shell sichern Sie die Ausgabe zusätzlich in den Arbeitsbereich, z. B.

```
udp_server.py --port 9999 | tee /node/udp-server.log
```

Beim Start über `u/t/i` landen die Ausgaben automatisch als Logdateien in `docker/out/`.
- Fenstergrößen und andere Werte aus `ss -tin` protokollieren Sie über die Zeit z. B. mit einer kleinen Schleife:

```
while true; do date +%s.%N >> /node/ss.log; \  
  ss -tin >> /node/ss.log; sleep 0.5; done
```

Die Datei `ss.log` finden Sie danach unter `docker/nodes/<host>/`.
- Für die Analyse können Wireshark, `tcptrace` oder ähnliche Werkzeuge hilfreich sein. `tcptrace` installieren Sie auf Ihrem Analyse-Rechner (Host), nicht in den Lab-Containern.
- Besonders genaues Timing ist wichtig für Jitter-Analysen.
- Falls nach einem Absturz Container übrig bleiben und sich das Lab beim nächsten Start seltsam verhält, räumen Sie mit `docker compose -p cuil down` auf.

Abgabe:

- Alle PCAP-Dateien der durchgeführten Experimente, klar benannt und mit Angabe der jeweiligen Netzwerkeinstellungen (Verlust, Jitter, Aufrufparameter der Werkzeuge).
- Ihre `lab.yaml` bzw. die eingestellten Linkparameter oder alternativ ein Screenshot des `cuil-lab`-Dashboards mit den eingestellten Linkeigenschaften.
- Ein ausführlicher schriftlicher Bericht, der die folgenden Punkte klar dokumentiert, analysiert und mit passenden Grafiken untermauert:
 - Verhalten von UDP unter normalen und verlustbehafteten Bedingungen, insbesondere Paketverluste und das Fehlen von Retransmissionsmechanismen.
 - Verhalten von TCP mit Fokus auf Retransmissions, Entwicklung des Empfangsfensters (`rwin`) und Flusskontrolle unter Verlustbedingungen.
 - Vergleich der TCP-Betriebsmodi (persistente Verbindung vs. Verbindungsaufbau pro Nachricht) hinsichtlich Overhead, Latenz und Stabilität.

- Auswirkungen von Jitter auf TCP-Flusskontrolle anhand von Messdaten.
- Beschreibung der eingesetzten Analysewerkzeuge (z. B. Wireshark, tcptrace) und der Auswertungsmethoden.
- Visuelle Aufarbeitung der Daten, z. B.:
 - * Paketverluste und Retransmissions als Zeitlinien- oder Paketverlaufdiagramme.
 - * Zeitlicher Verlauf des TCP-Empfangsfensters (rwin) als Diagramm.
 - * Paketankunftszeiten in Form von Zeitreihen oder Scatter-Plots zur Darstellung von Jitter.
 - * Statistische Auswertungen wie Paketverlustrate, durchschnittliche Latenz oder Retransmissionsrate.
 - * Vergleichende Darstellungen (Tabellen oder Grafiken) zu Performance-Eigenschaften der TCP-Modi.
- Die Grafiken müssen klar beschriftet und ausführlich interpretiert sein, sodass die Analyse für Dritte nachvollziehbar und verständlich bleibt.

Aufgabe 3 Vergleich von TCP Reno und TCP Cubic in cuil-lab mit iperf3, Wireshark und tcptrace

Ziel dieser Aufgabe ist es, das Verhalten und die Performance der TCP-Staukontrollalgorithmen **Reno** und **Cubic** in der **cuil-lab**-Umgebung zu untersuchen und zu vergleichen. Dazu konfigurieren Sie die jeweiligen Algorithmen auf den Hosts, erzeugen TCP-Verkehr mit iperf3, zeichnen den Netzwerkverkehr auf und analysieren diesen mit tcptrace und Wireshark.

Experiment-Setup:

Die Topologie steht bereit: zwei Hosts **n1** (Sender) und **n2** (Empfänger), über einen Shaper verbunden. Stellen Sie den Link über die Taste 1 auf folgende Eigenschaften ein:

- Bandbreite: 1 Mbit/s (1mbit)
- Verzögerung: 20 ms (20ms)
- Burst/Puffer: klein halten (Standardwert)
- Paketverlust: 3% (3%)

a) TCP Staukontrolle auf Reno einstellen und TCP Verkehr generieren

- Öffnen Sie mit **s** eine Shell auf **n1**. Prüfen Sie die verfügbaren TCP-Staukontrollalgorithmen mit:
`sysctl net.ipv4.tcp_available_congestion_control`
- Setzen Sie auf **n1** den Algorithmus auf **reno** (alternativ über die Taste **c** im Dashboard):
`sysctl -w net.ipv4.tcp_congestion_control=reno`
- Starten Sie auf **n2** iperf3 im Servermodus: `iperf3 -s`
- Starten Sie die Paketaufzeichnung mit **p** und stoppen Sie sie nach dem Testlauf wieder mit **p**. Es entstehen zwei PCAP-Dateien in `docker/out/` (`cap-<Zeitstempel>-n1.pcap` und `...-n2.pcap`). Für die tcptrace-Analyse verwenden Sie den senderseitigen Mitschnitt (`-n1`); benennen Sie ihn in `tcp_reno.pcap` um.
- Starten Sie auf **n1** den iperf3-Client zum Übertragen von TCP-Daten an **n2** (30 Sekunden Testdauer, 1 s Intervalle):
`iperf3 -c n2 -t 30 -i 1 > iperf3_reno_output.txt`
- Jede Shell startet im Arbeitsbereich `/node`; die Ausgabedatei liegt anschließend auf Ihrem Rechner unter `docker/nodes/n1/`.

b) TCP Staukontrolle auf Cubic wechseln und Test wiederholen

- Ändern Sie auf **n1** die Staukontrolle auf **cubic** (per Shell oder über die Taste **c**):
`sysctl -w net.ipv4.tcp_congestion_control=cubic`
- Wiederholen Sie die iperf3-Server- und Client-Starts sowie die Paketaufzeichnung analog zu Teilaufgabe (a), speichern Sie die senderseitige Aufzeichnung unter `tcp_cubic.pcap` und die iperf3-Ausgabe unter `iperf3_cubic_output.txt`.

c) Analyse der aufgezeichneten Pakete mittels tcptrace

- Falls nicht vorhanden, installieren Sie **tcptrace** auf dem Analyse-Rechner.
- Analysieren Sie die Aufzeichnungen `tcp_reno.pcap` und `tcp_cubic.pcap` mit **tcptrace**. Lesen Sie die Dokumentation für die Möglichkeiten: <https://linux.die.net/man/1/tcptrace>
- Erzeugen Sie mit **tcptrace** die Zeitverläufe der relevanten Kenngrößen (u. a. Staukontrollfenster) als Grundlage für den anschließenden Vergleich in Teilaufgabe (d).

d) Vergleich und Auswertung

- Vergleichen Sie die Durchsatzwerte aus den iperf3-Ausgaben für Reno und Cubic.
- Vergleichen Sie anhand der tcptrace-Auswertungen aus Teilaufgabe (c), wie sich das Staukontrollfenster bei beiden Algorithmen zeitlich verändert (z. B. Wachstumsmuster, Einbrüche bei Paketverlust).
- Identifizieren Sie Retransmissionen und deren Häufigkeit in den PCAP-Dateien mittels Wireshark.
- Diskutieren Sie, wie die unterschiedlichen Rhythmiken der Staukontrolle die TCP-Leistung beeinflussen.
- Begründen Sie, in welchen Netzwerkszenarien einer der beiden Algorithmen Vorteile haben könnte.

Abgabe:

- Gut kommentierter Quellcode etwaiger eigener Auswertungsskripte oder -Tools (z. B. Skripte zur Verarbeitung der PCAP-Dateien oder zur Erstellung von Grafiken). Die in der Umgebung enthaltenen Werkzeuge müssen Sie nicht erneut beilegen.
- Ihre `lab.yaml` bzw. die eingestellten Linkparameter oder alternativ ein Screenshot des `cuil-lab`-Dashboards mit den eingestellten Linkeigenschaften.
- Alle PCAP-Dateien der Experimente (`tcp_reno.pcap`, `tcp_cubic.pcap`), jeweils klar benannt und mit Angabe der jeweiligen Konfiguration.
- Alle iperf3-Ausgabedateien aus den Tests.
- Ein ausführlicher und strukturierter Bericht, der folgende Aspekte systematisch darstellt und analysiert:
 - Vergleich der Durchsatzwerte (iperf3) und deren zeitliche Entwicklung.
 - Analyse und grafische Darstellung des Staukontrollfensters (congestion window) beider Algorithmen über die Zeit (z. B. mittels tcptrace-Generierung).
 - Visualisierung und Interpretation der Retransmissionsmuster bei Reno und Cubic anhand der PCAP-Dateien (z. B. Paketverlauf, Häufigkeit).
 - Graphische Auswertung der Paketverluste und deren Einfluss auf das Staukontrollfenster.
 - Diskussion der Unterschiede im Verhalten der Staukontrolle: Wachstumsmuster, Reaktion auf Überlast (auch bei fehlendem Paketverlust), Stabilitätsaspekte.
 - Bewertung der Performance und Robustheit beider Algorithmen in Hinblick auf das gegebene Setup.
 - Begründete Abschätzung, in welchen praktischen Netzwerkszenarien Reno oder Cubic jeweils Vorteile bieten.
 - Beschreibung der verwendeten Analysewerkzeuge und Methoden (Wireshark, tcptrace, eigene Skripte), inklusive Erläuterung der Vorgehensweise zur Datenauswertung.
- Grafiken sollen dabei mindestens umfassen:
 - Zeitlicher Verlauf des congestion window für Reno und Cubic im Vergleich in einem Diagramm.
 - Durchsatzverlauf (iperf3-Output) über Testdauer grafisch dargestellt.
 - Visualisierung von Retransmissions und Paketverlusten als Zeitlinien oder als Histogramme.
- Alle Diagramme müssen beschriftet, mit Legenden versehen und ausführlich interpretiert werden, sodass die Analyse für Dritte nachvollziehbar ist.